VentaYa

Gestor de pedidos para pequeños comercios

PROYECTO FINAL

Programación 3 — Aplicaciones Móviles

Documentación: descripción, modelo de datos, casos de uso y navegación

Alumno	Facundo Neveu
Carrera	Tecnicatura en Desarrollo de Software — ITES
Docente	[completar]
Fecha	Noviembre de 2026
Repositorio GitHub	[completar]
API desplegada (Swagger)	http://ventaya.runasp.net/swagger

1. Descripción del proyecto

1.1 Idea

VentaYa es una aplicación móvil para pequeños comercios (kioscos, verdulerías, tiendas de ropa, gastronomía casera) que hoy reciben sus pedidos por WhatsApp y los anotan en cuadernos o planillas. Eso provoca pedidos olvidados, errores de precio, falta de control de stock y ninguna visión de cuánto se vende. VentaYa le da al dueño un panel simple, pensado para usarse con el pulgar, donde organiza su catálogo, gestiona pedidos, aplica cupones y costos de envío, y responde a sus clientes con mensajes prearmados.

1.2 Alcance

Incluye:

- Cuentas con inicio de sesión seguro (JWT) y dos roles, propietario y empleado; el propietario invita con un código.
- Gestión del negocio, de su catálogo (imágenes, precio, stock y activación) y de las zonas de envío.
- Pedidos cargados desde la app o enviados desde el catálogo público, con estados (Pendiente, Confirmado, Pagado, Entregado, Cancelado), descuento de stock y total calculado.
- Cupones de descuento (porcentaje o monto fijo, con mínimo, vencimiento y usos máximos) y respuestas rápidas con variables {nombre}, {total} y {link}.
- Panel de estadísticas con ventas del mes, pedidos por estado y uso del plan.
- Modelo freemium: plan Gratis con límites (10 productos, 30 pedidos por mes y 5 respuestas), prueba de 14 días y plan Pago sin límites. El cambio de plan está simulado: no hay cobro real.

No incluye (trabajo futuro): cobro online con MercadoPago, facturación electrónica, integración con la API de WhatsApp, logística de terceros, variantes de producto y notificaciones push.

1.3 Funcionalidad por actor

- **Propietario:** todo lo que hace el empleado, y además crea y edita su negocio, invita o revoca empleados y consulta o cambia su plan.
- **Empleado:** gestiona productos, pedidos, cupones, zonas de envío y respuestas rápidas, y consulta el panel. No accede al perfil del negocio, al equipo ni al plan.
- **Cliente final:** consulta el catálogo público del negocio y envía un pedido, con cupón y zona de envío, sin necesidad de crear una cuenta.

1.4 Arquitectura y tecnología

Tres capas: una app móvil .NET MAUI (Android) que consume una API REST en ASP.NET Core (.NET 10) con autenticación JWT y documentación Swagger, la cual persiste los datos en una base SQL Server de 12 tablas en tercera forma normal mediante Entity Framework Core. Backend y base de datos se despliegan en MonsterASP.NET. Las reglas de negocio cuentan con pruebas automáticas xUnit.

2. Modelo de datos — Diagrama Entidad-Relación

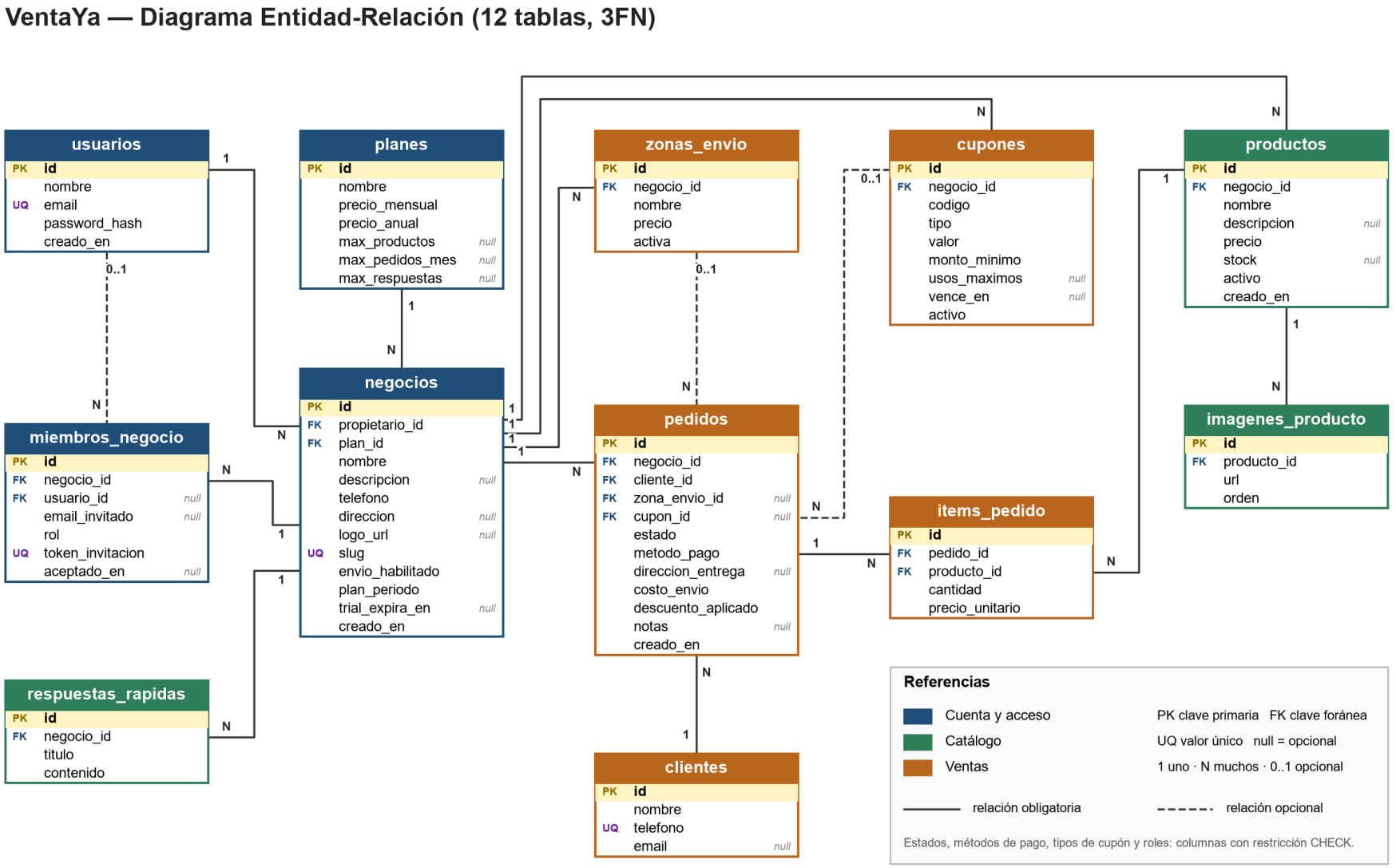


Figura 1. DER de VentaYa: 12 tablas normalizadas (3FN) para SQL Server.

2.1 Relaciones entre tablas

Relación (padre → hijo)	Clave foránea	Cardinalidad	Tipo
usuarios → negocios	negocios.propietario_id	1 a N	obligatoria
usuarios → miembros_negocio	miembros_negocio.usuario_id	0..1 a N	opcional
negocios → miembros_negocio	miembros_negocio.negocio_id	1 a N	obligatoria
planes → negocios	negocios.plan_id	1 a N	obligatoria
negocios → productos	productos.negocio_id	1 a N	obligatoria
negocios → zonas_envio	zonas_envio.negocio_id	1 a N	obligatoria
negocios → cupones	cupones.negocio_id	1 a N	obligatoria
negocios → respuestas_rapidas	respuestas_rapidas.negocio_id	1 a N	obligatoria
negocios → pedidos	pedidos.negocio_id	1 a N	obligatoria
productos → imagenes_producto	imagenes_producto.producto_id	1 a N	obligatoria
clientes → pedidos	pedidos.cliente_id	1 a N	obligatoria
zonas_envio → pedidos	pedidos.zona_envio_id	0..1 a N	opcional
cupones → pedidos	pedidos.cupon_id	0..1 a N	opcional
pedidos → items_pedido	items_pedido.pedido_id	1 a N	obligatoria
productos → items_pedido	items_pedido.producto_id	1 a N	obligatoria

2.2 Justificación de la normalización (3FN)

Primera forma normal (1FN). Todos los atributos son atómicos. Lo que sería una lista dentro de una fila (las imágenes de un producto o las líneas de un pedido) tiene su propia tabla: imagenes_producto e items_pedido.

Segunda forma normal (2FN). Todas las tablas usan una clave primaria simple (id), por lo que no existen dependencias parciales. Las unicidades compuestas, (negocio_id, codigo) en cupones y (negocio_id, usuario_id) en miembros_negocio, son restricciones de integridad y no claves de dependencia.

Tercera forma normal (3FN). Ningún atributo no clave depende de otro atributo no clave:

- **Los límites del plan viven en planes, no en negocios.** Así se evita la dependencia transitiva negocio → plan → límites.
- **Los datos del comprador viven en clientes** (teléfono → nombre), no repetidos en cada pedido. Un cliente puede comprar en varios negocios.
- **El total del pedido no se guarda.** Se calcula como suma de ítems + costo_envio − descuento_aplicado; guardarlo sería un dato derivado.
- **Los usos de un cupón tampoco se guardan:** se cuentan desde pedidos.
- **precio_unitario, costo_envio y descuento_aplicado sí se guardan,** porque son una foto histórica: si luego cambia el precio de un producto o de una zona, el pedido ya realizado no debe modificarse. No son derivables de las tablas de origen.
- **Estados, métodos de pago, tipos de cupón y roles** son dominios cerrados y pequeños: se modelan como columnas con restricción CHECK en lugar de tablas de pocas filas, para mantener el modelo en 12 tablas.

Integridad. Todas las relaciones tienen clave foránea. El borrado es RESTRICT por defecto, salvo imagenes_producto, items_pedido y miembros_negocio, que se eliminan en cascada con su padre. Un producto, cupón o zona ya usados en pedidos no se borra: se desactiva.

2.3 Diccionario de datos

usuarios

Personas que inician sesión en la app (propietarios y empleados).

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
nombre	nvarchar(100)	Obligatorio
email	nvarchar(200)	Único; Obligatorio
password_hash	nvarchar(100)	Obligatorio
creado_en	datetime2	Obligatorio

planes

Planes freemium (Gratis, Prueba, Pago) y sus límites. NULL = ilimitado.

Columna	Tipo (SQL Server)	Restricciones
id	int	Clave primaria; Obligatorio
nombre	nvarchar(30)	Obligatorio
precio_mensual	decimal(12,2)	Obligatorio
precio_anual	decimal(12,2)	Obligatorio
max_productos	int	Admite nulos
max_pedidos_mes	int	Admite nulos
max_respuestas	int	Admite nulos

negocios

Comercio del propietario. Su slug identifica el catálogo público.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
propietario_id	int	Clave foránea → usuarios.id; Obligatorio
plan_id	int	Clave foránea → planes.id; Obligatorio
nombre	nvarchar(100)	Obligatorio
descripcion	nvarchar(500)	Admite nulos
telefono	nvarchar(30)	Obligatorio
direccion	nvarchar(200)	Admite nulos
logo_url	nvarchar(300)	Admite nulos
slug	nvarchar(100)	Único; Obligatorio
envio_habilitado	bit	Obligatorio
plan_periodo	nvarchar(10)	Obligatorio
trial_expira_en	datetime2	Admite nulos
creado_en	datetime2	Obligatorio

miembros_negocio

Empleados de un negocio (relación N:M usuarios-negocios). usuario_id es NULL hasta aceptar la invitación.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
usuario_id	int	Clave foránea → usuarios.id; Admite nulos
email_invitado	nvarchar(200)	Admite nulos
rol	nvarchar(20)	Obligatorio
token_invitacion	nvarchar(64)	Único; Obligatorio
aceptado_en	datetime2	Admite nulos

productos

Productos del catálogo de un negocio. stock NULL = no se controla stock.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
nombre	nvarchar(120)	Obligatorio
descripcion	nvarchar(1000)	Admite nulos
precio	decimal(12,2)	Obligatorio
stock	int	Admite nulos
activo	bit	Obligatorio
creado_en	datetime2	Obligatorio

imagenes_producto

Imágenes de cada producto (hasta 5).

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
producto_id	int	Clave foránea → productos.id; Obligatorio
url	nvarchar(300)	Obligatorio
orden	int	Obligatorio

respuestas_rapidas

Plantillas de mensajes con variables {nombre}, {total} y {link}.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
titulo	nvarchar(100)	Obligatorio
contenido	nvarchar(1000)	Obligatorio

clientes

Compradores identificados por teléfono; pueden comprar en varios negocios.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
nombre	nvarchar(100)	Obligatorio
telefono	nvarchar(30)	Único; Obligatorio
email	nvarchar(200)	Admite nulos

zonas_envio

Zonas de entrega de un negocio con su costo.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
nombre	nvarchar(100)	Obligatorio
precio	decimal(12,2)	Obligatorio
activa	bit	Obligatorio

cupones

Cupones de descuento (porcentaje o monto fijo). Único por (negocio_id, codigo).

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
codigo	nvarchar(30)	Obligatorio
tipo	nvarchar(20)	Obligatorio
valor	decimal(12,2)	Obligatorio
monto_minimo	decimal(12,2)	Obligatorio
usos_maximos	int	Admite nulos
vence_en	datetime2	Admite nulos
activo	bit	Obligatorio

pedidos

Pedido de un cliente a un negocio. El total no se guarda: se calcula desde los ítems.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
negocio_id	int	Clave foránea → negocios.id; Obligatorio
cliente_id	int	Clave foránea → clientes.id; Obligatorio
zona_envio_id	int	Clave foránea → zonas_envio.id; Admite nulos
cupon_id	int	Clave foránea → cupones.id; Admite nulos
estado	nvarchar(20)	Obligatorio
metodo_pago	nvarchar(20)	Obligatorio
direccion_entrega	nvarchar(200)	Admite nulos
costo_envio	decimal(12,2)	Obligatorio
descuento_aplicado	decimal(12,2)	Obligatorio
notas	nvarchar(500)	Admite nulos
creado_en	datetime2	Obligatorio

items_pedido

Líneas de un pedido. precio_unitario es una foto del precio al momento de la compra.

Columna	Tipo (SQL Server)	Restricciones
id	int identity	Clave primaria; Obligatorio
pedido_id	int	Clave foránea → pedidos.id; Obligatorio
producto_id	int	Clave foránea → productos.id; Obligatorio
cantidad	int	Obligatorio
precio_unitario	decimal(12,2)	Obligatorio

3. Diagrama de casos de uso (UML)

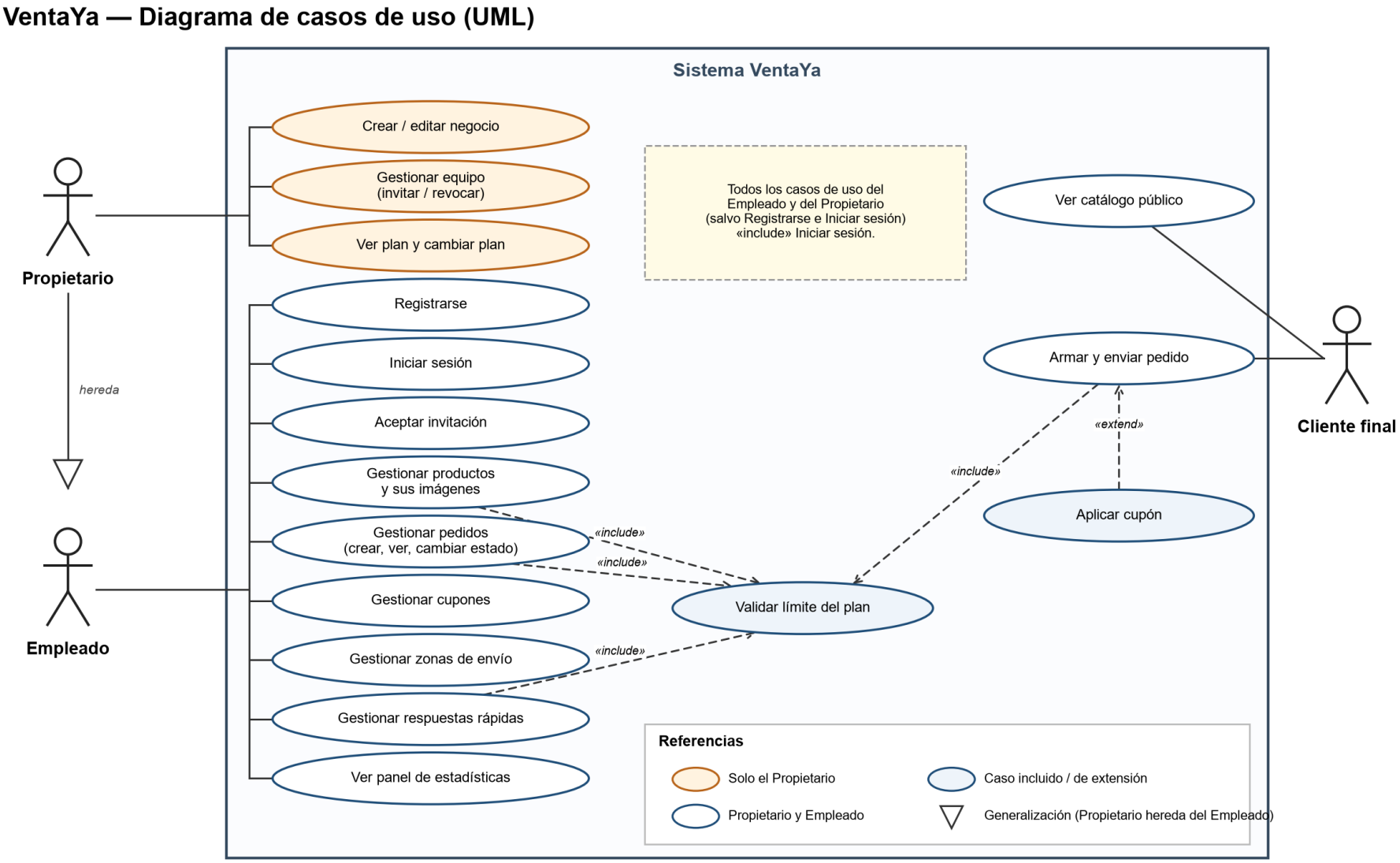


Figura 2. Actores y casos de uso de VentaYa.

3.1 Actores

- **Propietario:** dueño del comercio. Hereda todos los casos de uso del Empleado.
- **Empleado:** persona invitada por el propietario para operar el negocio.
- **Cliente final:** comprador que usa el catálogo público, sin cuenta.

3.2 Especificación de los casos de uso

ID	Caso de uso	Actor	Descripción	Endpoint
CU01	Registrarse	Propietario, Empleado	Crea una cuenta con nombre, email y contraseña (se guarda cifrada).	POST /api/auth/registro
CU02	Iniciar sesión	Propietario, Empleado	Valida las credenciales y entrega un token JWT que la app usa en cada pedido.	POST /api/auth/login · GET /api/auth/yo
CU03	Crear / editar negocio	Propietario	Da de alta el negocio (inicia la prueba de 14 días) y edita sus datos y la opción de envíos.	POST · PUT /api/negocio
CU04	Gestionar productos y sus imágenes	Propietario, Empleado	Alta, edición, baja o desactivación. Hasta 5 imágenes por producto. Incluye CU13.	/api/productos
CU05	Gestionar pedidos	Propietario, Empleado	Cargar un pedido, listarlos por estado, ver el detalle y cambiar su estado. Valida transiciones, descuenta stock y lo devuelve al cancelar. Incluye CU13.	/api/pedidos
CU06	Gestionar cupones	Propietario, Empleado	Cupones por porcentaje o monto fijo, con mínimo, vencimiento y usos máximos.	/api/cupones
CU07	Gestionar zonas de envío	Propietario, Empleado	Zonas de entrega con su costo; se pueden activar o desactivar.	/api/zonas-envio
CU08	Gestionar respuestas rápidas	Propietario, Empleado	Plantillas de mensajes con variables {nombre}, {total}, {link}. Incluye CU13.	/api/respuestas-rapidas
CU09	Ver panel de estadísticas	Propietario, Empleado	Ventas del mes y totales, pedidos por estado y uso del plan.	GET /api/panel
CU10	Gestionar equipo	Propietario	Invitar empleados (se genera un código), listar y revocar el acceso.	/api/equipo
CU11	Aceptar invitación	Empleado	Con el código recibido, el usuario pasa a operar el negocio del propietario.	POST /api/equipo/aceptar/{token}
CU12	Ver plan y cambiar plan	Propietario	Muestra límites y uso. El cambio Gratis/Pago es simulado (sin cobro).	PUT /api/negocio/plan
CU13	Validar límite del plan	Sistema (incluido)	Rechaza altas que superen los límites del plan Gratis. Con la prueba vigente o el plan Pago no hay límites.	(interno)
CU14	Ver catálogo público	Cliente final	Consulta productos activos y zonas de envío de un negocio por su slug.	GET /api/publico/{slug}
CU15	Armar y enviar pedido	Cliente final	Envía el pedido con sus datos, zona de envío y método de pago. El precio siempre se toma de la base. Incluye CU13.	POST /api/publico/{slug}/pedidos
CU16	Aplicar cupón	Cliente final (extiende CU15)	Valida el cupón (activo, vigente, mínimo y usos) y descuenta el importe del pedido.	(dentro de CU15)

4. Diagrama de navegación de pantallas

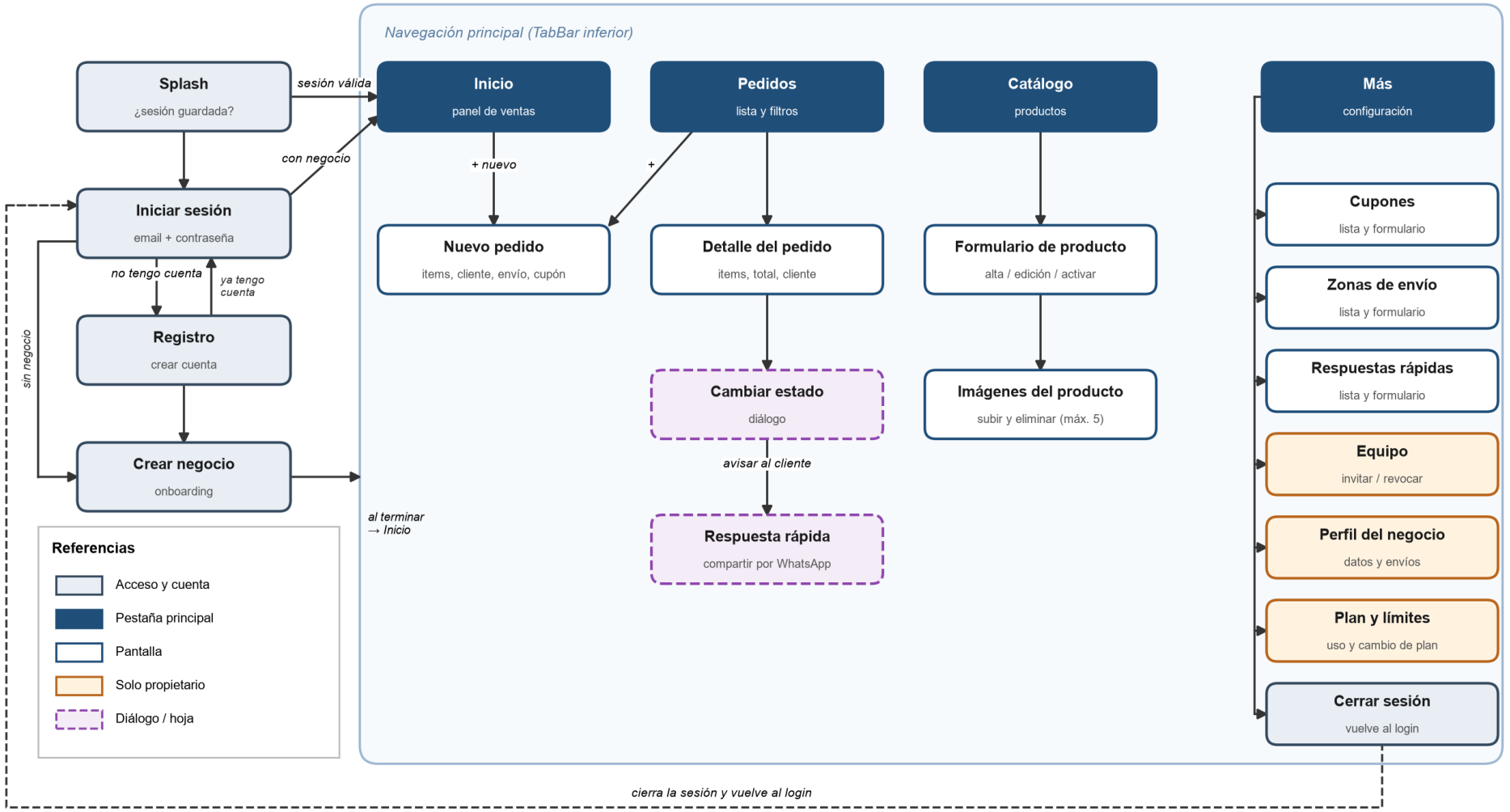


Figura 3. Flujo de pantallas de la app móvil (.NET MAUI).

4.1 Descripción de las pantallas

Pantalla	Qué hace	Endpoint principal
Splash	Decide a dónde ir: con sesión válida abre Inicio; si no, Iniciar sesión.	GET /api/auth/yo
Iniciar sesión	Email y contraseña. Con negocio va a Inicio; sin negocio, a Crear negocio.	POST /api/auth/login
Registro	Alta de cuenta (nombre, email, contraseña).	POST /api/auth/registro
Crear negocio	Onboarding: nombre, teléfono, dirección y si hace envíos.	POST /api/negocio
Inicio	Ventas del mes, pedidos por estado, últimos pedidos y uso del plan.	GET /api/panel
Pedidos	Lista con filtro por estado.	GET /api/pedidos
Nuevo pedido	Elegir productos y cantidades, cliente, zona de envío, cupón y método de pago.	POST /api/pedidos
Detalle del pedido	Ítems, totales y datos del cliente.	GET /api/pedidos/{id}
Cambiar estado	Diálogo con los estados permitidos desde el actual.	PATCH /api/pedidos/{id}/estado
Respuesta rápida	Elige una plantilla, completa las variables y la comparte por WhatsApp.	GET /api/respuestas-rapidas/{id}/renderizada
Catálogo	Lista y búsqueda de productos.	GET /api/productos
Formulario de producto	Alta, edición, activación o baja de un producto.	POST · PUT · DELETE /api/productos
Imágenes del producto	Subir o eliminar imágenes (máx. 5).	POST · DELETE /api/productos/{id}/imagenes
Cupones	Lista y formulario de cupones.	/api/cupones
Zonas de envío	Lista y formulario de zonas.	/api/zonas-envio
Respuestas rápidas	Lista y formulario de plantillas.	/api/respuestas-rapidas
Equipo (propietario)	Invitar empleados y revocar accesos.	/api/equipo
Perfil del negocio (propietario)	Editar los datos del negocio.	PUT /api/negocio
Plan y límites (propietario)	Uso del plan y cambio simulado.	PUT /api/negocio/plan
Cerrar sesión	Borra el token guardado y vuelve a Iniciar sesión.	(local)

Nota: el empleado no ve las pantallas marcadas como «propietario» (Equipo, Perfil del negocio y Plan y límites).

5. API REST, despliegue y pruebas de ejecución

5.1 Despliegue

Componente	Tecnología	Ubicación
App móvil	.NET MAUI (Android)	APK / ejecución en emulador o dispositivo
API REST	ASP.NET Core (.NET 10) + Swagger	MonsterASP: ventaya.runasp.net
Base de datos	SQL Server (Entity Framework Core)	MonsterASP (crea el esquema al iniciar)
Código fuente	Git	GitHub — [completar URL]

5.2 Pruebas de ejecución

- **Automáticas (xUnit):** 67 pruebas: reglas de negocio y servicios (cupones, estados, límites del plan, stock) y la lógica de las pantallas de la app contra la API real en memoria. Se ejecutan con «dotnet test».
- **Postman / Newman:** 75 requests y 124 aserciones (docs/postman), 0 fallas en local y en MonsterASP.
- **Swagger:** documentación interactiva en /swagger, con autenticación JWT.
- **Evidencia de la app móvil:** [completar con capturas o video]